

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Trabajo Práctico Integrador

Materia: Programación 1

Tema: Aplicación de listas, diccionarios, funciones y archivos CSV

Integrantes: Crespi, Claudio Sebastian y Doglioli Nicolas

Comisión: 10-11

Fecha de entrega: 15/06/2026

Índice

1. Introducción
2. Objetivo y alcance
3. Consigna aplicada al proyecto
4. Estructura del sistema
5. Modelo de datos
6. Funcionalidades implementadas
7. Validaciones y manejo de errores
8. Decisiones técnicas y arquitectura
9. Dificultades y conclusiones
10. Links

1. Introducción

Este trabajo práctico integrador desarrolla un sistema de gestión de países en Python orientado a consola. El proyecto toma como base un archivo CSV con datos de países y permite consultar, ordenar, filtrar, actualizar y agregar información utilizando estructuras de datos simples y funciones modulares.

La implementación se apoya principalmente en listas de diccionarios, validaciones de entrada y operaciones sobre archivos. El resultado es una aplicación pensada para practicar los contenidos centrales de Programación 1 con un problema concreto y cercano al tratamiento de datos.

2. Objetivo y alcance

El objetivo general es cumplir con la consigna del TPI mediante una aplicación capaz de leer un dataset desde CSV, ofrecer un menú interactivo y generar información útil a partir de los datos cargados.

En el código de referencia se observa un alcance funcional completo: alta de países, actualización de valores, búsquedas por nombre, filtros por distintos criterios, ordenamientos y estadísticas básicas. También se contemplan controles de formato para evitar errores de ejecución frente a entradas inválidas.

3. Consigna aplicada al proyecto

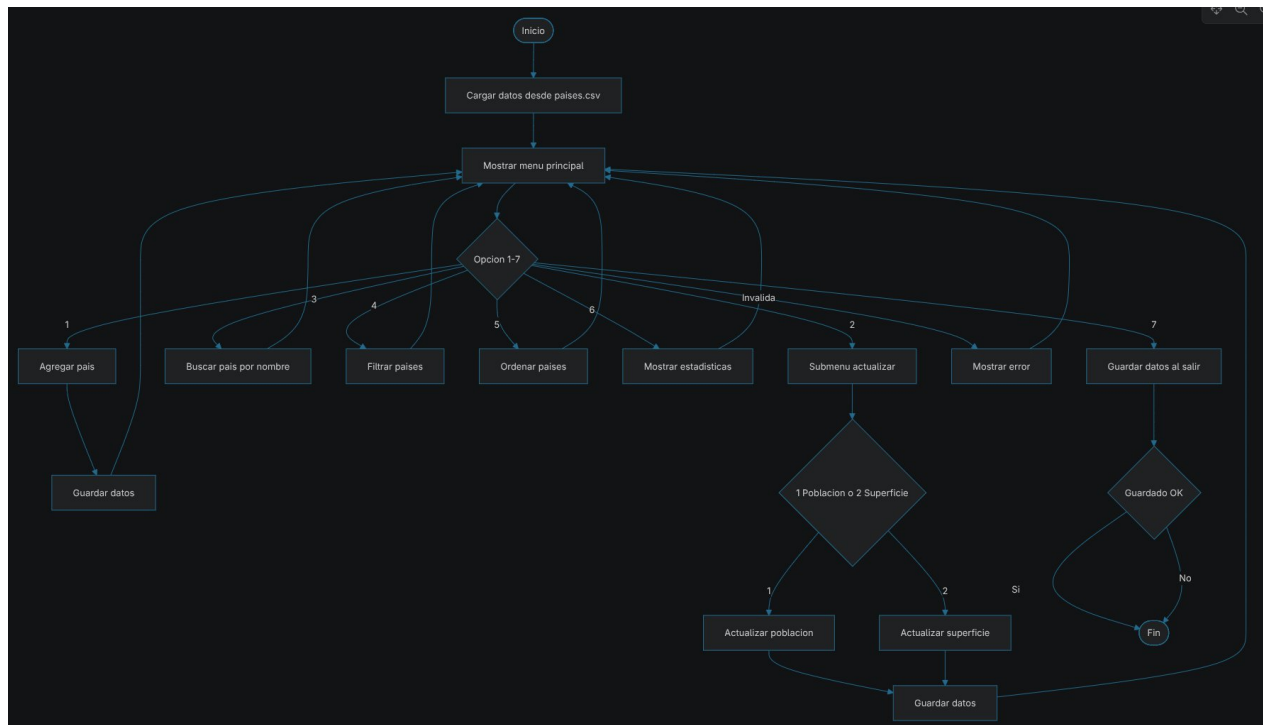
La consigna solicita una aplicación en Python 3.x que use listas, diccionarios, funciones, condicionales, estructuras repetitivas y archivos CSV. Además, pide que el sistema sea modular, claro y con validaciones de entrada.

El proyecto analizado responde a esos requisitos con dos archivos principales: uno dedicado al menú y flujo general del programa, y otro concentrado en las funciones de carga, validación y tratamiento de datos. El archivo `países.csv` actúa como persistencia simple para conservar la información entre ejecuciones.

4. Estructura del sistema

Archivo	Rol principal
<code>tpi.py</code>	Define el menú principal, el bucle de ejecución y la coordinación entre funciones, Incluye carga/guardado CSV, validaciones y operaciones sobre la lista de países.
<code>países.csv</code>	Base de datos simple con el dataset de países.
<code>README.md</code>	Describe el programa e instrucciones de uso.

Esta separación favorece la legibilidad y evita mezclar la interfaz de consola con la lógica de datos.



5. Modelo de datos

Cada país se representa como un diccionario con cuatro campos: nombre, poblacion, superficie y continente. Esa decisión simplifica el acceso a cada atributo y hace posible trabajar con una lista de registros homogéneos.

Ejemplo conceptual de un registro:

```
{
    'nombre': 'Argentina',
    'poblacion': 45376763,
    'superficie': 2780400,
    'continente': 'América'
}
```

6. Funcionalidades implementadas

El programa ofrece un menú de consola que permite ejecutar las operaciones exigidas por la consigna:

- Agregar un país validando que no existan campos vacíos y que población y superficie sean números positivos.
- Actualizar población o superficie de un país existente.
- Buscar países por coincidencia exacta o parcial del nombre.
- Filtrar por continente, rango de población o rango de superficie.
- Ordenar por nombre, población o superficie en forma ascendente o descendente.

- Calcular estadísticas: mayor y menor población, promedio de población, promedio de superficie y cantidad de países por continente.

La función main actúa como punto de entrada y mantiene un ciclo permanente hasta que el usuario decide guardar y salir. Cada opción del menú delega el trabajo en una función específica, manteniendo el código ordenado y predecible.

7. Validaciones y manejo de errores

Las validaciones son una parte central del diseño. El programa controla que los textos no estén vacíos, que los valores numéricos sean enteros positivos y que los criterios de filtro o búsqueda tengan sentido. En caso de error, muestra mensajes claros para guiar al usuario.

- Carga del CSV: si el archivo no existe o presenta datos inválidos, el sistema inicia con una lista vacía y notifica el problema.
- Alta de país: no permite nombres duplicados, ni población o superficie menores o iguales a cero.
- Actualización: verifica que el país exista antes de modificar sus datos.
- Filtros y búsquedas: evita entradas vacías o mal formadas y responde de manera controlada cuando no hay resultados.

8. Decisiones técnicas y arquitectura

La solución prioriza funciones pequeñas y con una responsabilidad concreta. Esa modularización facilita la lectura, la depuración y una futura extensión del sistema.

Las decisiones más relevantes fueron:

1. Usar una lista de diccionarios para representar la colección de países.
2. Separar la persistencia CSV de la lógica del menú.
3. Centralizar las validaciones para reutilizarlas en distintas operaciones.
4. Usar sorted para los ordenamientos y recorridos simples para filtros y estadísticas.
5. Guardar automáticamente luego de las operaciones de alta y actualización para no perder cambios.

Flujo general del sistema:

1. Se carga el CSV al iniciar el programa.
2. Se muestra el menú principal en consola.
3. El usuario elige una operación.
4. La función correspondiente procesa los datos y valida entradas.
5. Si corresponde, se guardan los cambios en el CSV.
6. El ciclo continúa hasta que el usuario selecciona guardar y salir.

9. Dificultades

Durante el desarrollo del proyecto, la principal complicación radicó en la falta de una arquitectura de datos unificada desde la fase de planificación. Si bien se distribuyeron de manera clara las funciones que cada integrante debía programar, se omitió consensuar el mecanismo metodológico para la carga, manipulación y persistencia de la información en el archivo de almacenamiento secundario (CSV).

Esta falta de estandarización inicial derivó en el planteamiento de dos estrategias técnicas divergentes dentro del equipo de trabajo:

- **Enfoque de persistencia iterativa:** Propuesto por el integrante Nicolás, consistía en realizar la apertura, lectura, modificación y cierre del archivo CSV de forma localizada dentro de cada función. El procesamiento interno se diseñó mediante la iteración secuencial de listas.
- **Enfoque de centralización en memoria dinámica:** Propuesto por el integrante Sebastián, planteaba realizar una lectura única al inicio de la ejecución para migrar los datos a una estructura de listas con diccionarios. Las modificaciones se operarían directamente en la memoria RAM, ejecutando la escritura final en el archivo CSV exclusivamente cuando el usuario manifestara la intención de finalizar la sesión.

Tras un análisis comparativo, el equipo determinó que, aunque ambos enfoques satisfacían el requerimiento funcional, el método de centralización en memoria presentaba ventajas técnico-estructurales superiores. Al reducir la interacción con el archivo a solo dos eventos (lectura inicial y escritura final), se minimizó el consumo de recursos del sistema (operaciones de I/O) y se mitigó el riesgo de corrupción de datos asociado a la apertura y cierre reiterado del archivo. Asimismo, esta elección derivó en un código más compacto, limpio y de menor complejidad algorítmica.

Conclusiones

El desarrollo de este trabajo integrador permitió consolidar los conocimientos técnicos adquiridos y comprender la relevancia de las buenas prácticas en la ingeniería de software. La complicación surgida en torno a la gestión del flujo de datos evidenció una limitación metodológica propia de la falta de experiencia previa; sin embargo, se transformó en una de las lecciones aprendidas más significativas del proyecto.

El proceso demostró que la planificación temprana de la arquitectura de datos —y no solo la división de tareas individuales— es un factor crítico para el éxito del desarrollo de software. La resolución del conflicto mediante el debate técnico y la evaluación de eficiencia (gasto de recursos y mitigación de errores) resalta el valor del trabajo en equipo y el criterio analítico. En última instancia, el proyecto no solo cumplió de manera óptima con los objetivos funcionales planteados, sino que sentó las bases metodológicas para afrontar futuros desarrollos sistémicos con una visión más estructurada, eficiente y profesional.

10. Links

[github](#)

[youtube](#)